VARDHAMAN
COLLEGE OF ENGINEERING

An Internship Project Report on

Verilog-Based UART Transmitter with Configurable Baud Rate Generator

ePBL Internship Program 2026

Domain: VLSI

Submitted by

K. Keerthika 23881A0422
K. Karthik 23881A0426
K. Aravind 23881A0427

Supervisor

Dr. Babu I
Assistant Professor
Department of Electronics and Communication Engineering
Vardhaman College of Engineering, Hyderabad

May 2026

Contents

Abstract

The design and implementation of a Verilog-Based UART (Universal Asynchronous Receiver-Transmitter) Transmitter with a Configurable Baud Rate Generator for reliable asynchronous serial communication in digital systems. The transmitter is developed using Verilog HDL and follows the standard 8N1 UART protocol, consisting of one start bit, eight data bits, no parity bit, and one stop bit. A configurable baud rate generator is incorporated to support multiple communication speeds by adjusting the clock division factor, making the design flexible for various embedded and FPGA-based applications. The system is modularly designed with separate transmitter control logic, baud rate generator, and shift register modules, improving readability, scalability, and ease of verification. Functional simulation is carried out to verify the correct generation of UART frames and accurate serial data transmission at different baud rates. The proposed design demonstrates efficient hardware utilization, reliable data transfer, and portability across different digital platforms. This project provides a practical understanding of serial communication protocols, digital design using Verilog HDL, and hardware implementation techniques, making it suitable for FPGA-based communication systems and embedded applications.

Keywords: Route Verilog,Baud Rate Generator, asynchronous serial communication, FPGA, Shortest Path.

1 Introduction

Serial communication plays a crucial role in modern digital and embedded systems by enabling efficient data exchange between electronic devices. Among the various serial communication protocols, the Universal Asynchronous Receiver-Transmitter (UART) is one of the most widely used due to its simplicity, low hardware requirements, and reliable performance. UART enables asynchronous communication, meaning that data can be transmitted without the need for a shared clock signal between the transmitting and receiving devices. This makes it an ideal choice for applications such as microcontrollers, sensors, GPS modules, Bluetooth devices, computers, and FPGA-based systems.

A UART transmitter converts parallel data into a serial bit stream by appending a start bit and a stop bit according to the UART communication protocol. The accuracy of data transmission depends largely on the baud rate, which defines the speed of communication in bits per second. Supporting multiple baud rates increases the flexibility and compatibility of the communication system with different devices.

This project focuses on the design and implementation of a Verilog-Based UART Transmitter with a Configurable Baud Rate Generator. The transmitter is designed using Verilog Hardware Description Language (HDL) following the standard 8N1 UART format, which consists of one start bit, eight data bits, no parity bit, and one stop bit. A configurable baud rate generator is integrated into the design, allowing the communication speed to be modified by changing the clock division factor. This feature makes the design reusable for different system clock frequencies and communication requirements.

The entire design follows a modular approach, where the baud rate generator, transmitter control logic, and shift register are implemented as separate modules. This improves readability, simplifies debugging, and allows future enhancements such as the addition of parity checking or a UART receiver. Functional verification is performed through simulation to ensure correct serial data transmission, proper frame generation, and accurate timing at different baud rates.

The project provides practical exposure to digital system design, finite state machines, serial communication protocols, and hardware description languages. It also serves as a foundation for implementing more advanced communication interfaces in FPGA and ASIC-based embedded systems.

2 Literature Review

The Universal Asynchronous Receiver-Transmitter (UART) is a widely used serial communication protocol in embedded systems due to its simplicity, reliability, and low hardware requirements. Unlike synchronous communication, UART does not require a shared clock signal, making it suitable for communication between microcontrollers, computers, sensors, and FPGA-based systems.

Many UART transmitter designs have been implemented using Verilog HDL, following the standard 8N1 format with one start bit, eight data bits, no parity bit, and one stop bit. Most designs include a baud rate generator, control logic, and shift register to ensure accurate serial data transmission.

Recent research focuses on configurable baud rate generators, which allow the communication speed to be changed by modifying the clock division factor. This improves flexibility and enables the same UART design to support multiple devices operating at different baud rates. Modular design approaches have also become common, making the system easier to understand, verify, and expand.

Based on these studies, this project implements a Verilog-Based UART Transmitter with a Configurable Baud Rate Generator using a modular architecture. The design provides reliable serial communication, flexible baud rate selection, and efficient hardware implementation,

making it suitable for FPGA-based embedded applications.

3 Objectives

- To design and implement a UART transmitter using Verilog HDL.
- To develop a configurable baud rate generator that supports multiple communication speeds.
- To implement the standard 8N1 UART communication protocol for reliable serial data transmission.
- To verify the functionality of the design through simulation and waveform analysis.
- To create a modular and scalable architecture suitable for FPGA-based applications.
- To achieve efficient hardware utilization while ensuring accurate and reliable data transmission.

4 Methodology

4.1 Block Diagram

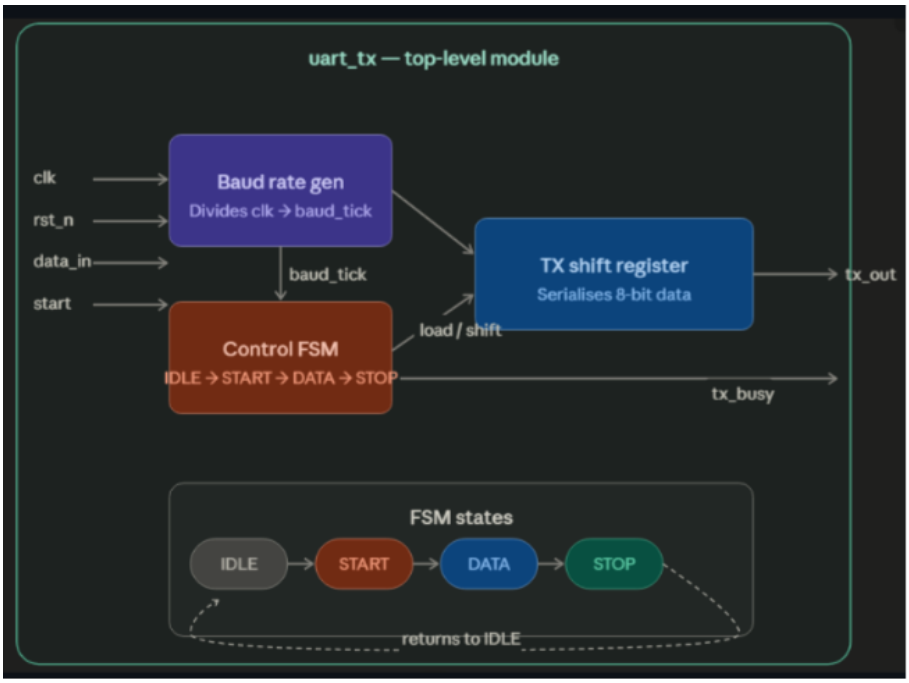

Figure 1: Block diagram

The block diagram as shown in Fig. 1 the overall architecture of the Verilog-Based UART Transmitter with Configurable Baud Rate Generator. The system accepts parallel input data and a system clock as inputs. The Baud Rate Generator produces the timing pulses required for serial communication at the selected baud rate. The UART Transmitter uses these timing signals to convert the parallel data into a serial data stream following the standard 8N1 UART protocol. Finally, the serial output (TX) is transmitted to the receiving device. The modular structure of the design improves flexibility, simplifies verification, and allows easy expansion for future enhancements such as parity generation or a UART receiver.

4.2 Working Principle

The proposed UART transmitter operates by transmitting digital data serially using a configurable communication speed. Initially, the system remains in the idle state, where the transmission line stays at logic high. When a transmission request is received, the UART transmitter loads the 8-bit input data into an internal shift register.

The baud rate generator continuously divides the system clock to generate timing pulses required for serial communication. These pulses determine the duration of each transmitted bit and ensure that the selected baud rate is maintained throughout the transmission process.

Once transmission begins, the UART transmitter sends a start bit, followed by the eight data bits in Least Significant Bit (LSB) first order. After all data bits have been transmitted, a stop bit is generated to indicate the completion of the data frame. The serial output is then returned to the idle state until the next transmission request is received.

A Finite State Machine (FSM) supervises the entire operation by controlling the sequence of transmission states and synchronizing data transfer with the baud rate generator. This coordinated operation ensures accurate timing, reliable serial communication, and efficient hardware implementation. The modular design also allows the baud rate to be easily modified without changing the transmitter architecture, making the system suitable for a wide range of FPGA and embedded communication applications.

This version focuses on how the circuit works step by step, making it different from the Introduction and Literature Review, which discuss the concept and background rather than the actual operation.

4.3 Software Implementation

The proposed UART transmitter was designed and verified using the EDA Playground, an online platform for developing and simulating Hardware Description Language (HDL) designs. The project was implemented using Verilog HDL, where the UART transmitter, baud rate generator, and testbench were written as separate modules to maintain a modular and organized design.

The simulation was carried out using the built-in simulator available in EDA Playground. A testbench was created to provide clock signals, reset, transmit enable, and input data to the UART transmitter. The simulation results were analyzed through waveform outputs to verify the correct generation of the start bit, data bits, and stop bit according to the selected baud rate.

EDA Playground provided a convenient environment for writing, debugging, and testing the Verilog code without requiring local software installation. The generated waveforms confirmed that the UART transmitter functioned correctly and successfully converted parallel input data into serial output following the standard 8N1 UART protocol.s input validation to prevent invalid city names and ensures smooth execution for all supported operations.

5 Verilog Code

5.1 design.v

```
// baud_gen.v
// Divides system clock to produce a 1-cycle pulse at baud rate
// For 50 MHz clk and 9600 baud: DIVISOR = 50_000_000 / 9600 = 5208

module baud_gen #(
    parameter DIVISOR = 5208
)(
    input  wire clk,
    input  wire rst_n,
    output reg  baud_tick
);
    integer cnt;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            cnt       <= 0;
            baud_tick <= 0;
        end else if (cnt == DIVISOR - 1) begin
            cnt       <= 0;
            baud_tick <= 1;
        end else begin
            cnt       <= cnt + 1;
            baud_tick <= 0;
        end
    end
endmodule

// uart_tx.v
// UART 8N1 transmitter: 8 data bits, no parity, 1 stop bit

module uart_tx #(
    parameter DIVISOR = 5208
)(
    input  wire       clk,
    input  wire       rst_n,
    input  wire [7:0] data_in,
    input  wire       start,
    output reg        tx_out,
    output reg        tx_busy
);

    // ---------- Baud Rate Generator ----------
    wire baud_tick;
    baud_gen #(.DIVISOR(DIVISOR)) u_baud (
        .clk      (clk),
        .rst_n    (rst_n),
        .baud_tick(baud_tick)
    );

    // ---------- FSM ----------
    localparam IDLE  = 2'd0,
               START = 2'd1,
               DATA  = 2'd2,
               STOP  = 2'd3;
```

```
55
56      reg [1:0] state;
57      reg [7:0] shift_reg;
58      reg [2:0] bit_cnt;   // counts 0..7
59
60      always @(posedge clk or negedge rst_n) begin
61          if (!rst_n) begin
62              state     <= IDLE;
63              tx_out    <= 1'b1;   // idle line is HIGH
64              tx_busy   <= 1'b0;
65              shift_reg <= 8'd0;
66              bit_cnt   <= 3'd0;
67          end else begin
68              case (state)
69
70                  IDLE: begin
71                      tx_out  <= 1'b1;
72                      tx_busy <= 1'b0;
73                      if (start) begin
74                          shift_reg <= data_in;
75                          tx_busy   <= 1'b1;
76                          state     <= START;
77                      end
78                  end
79
80                  START: begin
81                      if (baud_tick) begin
82                          tx_out  <= 1'b0;   // start bit
83                          bit_cnt <= 3'd0;
84                          state   <= DATA;
85                      end
86                  end
87
88                  DATA: begin
89                      if (baud_tick) begin
90                          tx_out    <= shift_reg[0];   // LSB first
91                          shift_reg <= shift_reg >> 1;
92                          if (bit_cnt == 3'd7)
93                              state <= STOP;
94                          else
95                              bit_cnt <= bit_cnt + 1;
96                      end
97                  end
98
99                  STOP: begin
100                     if (baud_tick) begin
101                         tx_out  <= 1'b1;   // stop bit
102                         state   <= IDLE;
103                     end
104                 end
105
106             endcase
107         end
108     end
109 endmodule
```

5.2 testbench.v

```
// tb_uart_tx.v
`timescale 1ns/1ps

module tb_uart_tx;

    localparam DIVISOR  = 16;
    localparam CLK_HALF = 5;

    reg        clk, rst_n, start;
    reg  [7:0] data_in;
    wire       tx_out, tx_busy;

    uart_tx #(.DIVISOR(DIVISOR)) dut (
        .clk    (clk),
        .rst_n  (rst_n),
        .data_in(data_in),
        .start  (start),
        .tx_out (tx_out),
        .tx_busy(tx_busy)
    );

    always #CLK_HALF clk = ~clk;

    // ---------- Receiver ----------
    reg [7:0] rx_byte;
    integer   i;

    task receive_byte;
        integer j;
        begin
            @(negedge tx_out);
            repeat(DIVISOR / 2) @(posedge clk);
            rx_byte = 8'd0;
            for (j = 0; j < 8; j = j + 1) begin
                repeat(DIVISOR) @(posedge clk);
                rx_byte[j] = tx_out;
            end
            repeat(DIVISOR) @(posedge clk);
        end
    endtask

    // ---------- Test ----------
    integer pass_count;

    task send_and_check;
        input [7:0] byte_to_send;
        begin
            data_in = byte_to_send;
            start   = 1;
            @(posedge clk); #1;
            start = 0;
            receive_byte;
            if (rx_byte === byte_to_send) begin
                $display("PASS: sent=0x%02h  received=0x%02h",
                   byte_to_send, rx_byte);
                pass_count = pass_count + 1;
```

```
            end else begin
                $display("FAIL: sent=0x%02h  received=0x%02h",
                    byte_to_send, rx_byte);
            end
        end
    endtask

    initial begin
        clk        = 0;
        rst_n      = 0;
        start      = 0;
        data_in    = 8'd0;
        pass_count = 0;

        repeat(4) @(posedge clk);
        rst_n = 1;
        repeat(2) @(posedge clk);

        send_and_check(8'h41);
        send_and_check(8'hFF);
        send_and_check(8'h00);
        send_and_check(8'h55);
        send_and_check(8'hAA);
        send_and_check(8'h5A);

        $display("----------------------------");
        $display("Tests passed: %0d / 6", pass_count);
        $display("----------------------------");
        $finish;
    end

    initial begin
        #5_000_000;
        $display("TIMEOUT");
        $finish;
    end

    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(0, tb_uart_tx);
    end

endmodule
```

6 Results and Discussion

The proposed Verilog-Based UART Transmitter with Configurable Baud Rate Generator was successfully implemented and verified using EDA Playground. Functional simulation confirmed that the transmitter correctly converted 8-bit parallel input data into a serial output following the standard 8N1 UART protocol. The generated waveform as shown in Fig. 2 clearly showed the transmission sequence consisting of a start bit, eight data bits (LSB first), and a stop bit.

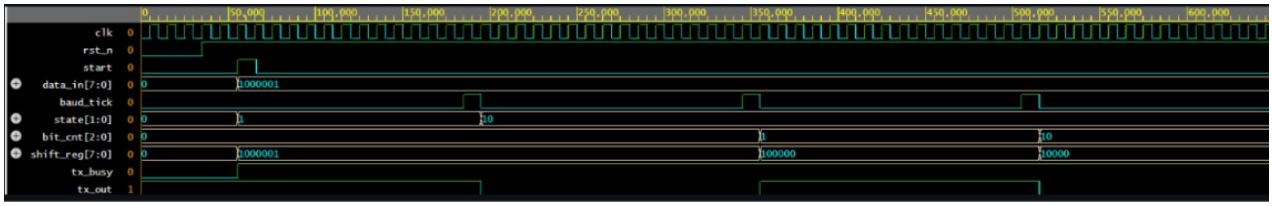

Figure 2: Output Waveform

The configurable baud rate generator produced accurate timing pulses, allowing the transmitter to operate at different communication speeds by modifying the clock division factor. The modular architecture simplified the verification process and ensured that each module functioned correctly before integration.

The simulation results demonstrated reliable serial data transmission with proper bit timing and correct frame generation. Overall, the design achieved the intended functionality while maintaining simplicity, flexibility, and efficient hardware implementation, making it suitable for FPGA-based and embedded serial communication applications.

7 Program Outcomes (POs)

Table 1: Program Outcomes Mapping

PO	**Contribution to the Project**
PO1	Applied knowledge of digital electronics and Verilog HDL to design and implement a UART transmitter.
PO2	Analyzed the UART communication protocol and developed a configurable baud rate generator for reliable data transmission.
PO3	Designed and verified a modular hardware architecture for efficient serial communication.
PO4	Conducted functional testing and analyzed simulation waveforms to validate the design.
PO5	Utilized modern engineering tools such as EDA Playground for coding, simulation, and verification.
PO12	Recognized the importance of continuous learning by exploring FPGA-based digital communication system design.

8 Sustainable Development Goals (SDGs)

Table 2: Sustainable Development Goals (SDGs) Mapping

SDG	**Contribution to the Project**
SDG 7	Affordable and Clean Energy-The modular Verilog implementation and configurable baud rate generator enable efficient hardware utilization, which can reduce power consumption in FPGA-based communication systems. Energy-efficient digital circuits contribute to sustainable electronic system design
SDG 9	Industry, Innovation and Infrastructure – The project develops a UART transmitter for reliable serial communication in FPGA and embedded systems. Such communication modules are fundamental components of industrial automation, IoT devices, robotics, and digital infrastructure, supporting technological innovation and modern industries.
SDG 11	Sustainable Cities and Communities – UART communication is widely used in smart city applications, including traffic control systems, environmental monitoring, smart meters, and public safety devices. The proposed transmitter can serve as a communication interface in such embedded applications.
SDG 12	Responsible Consumption and Production-The reusable and configurable design minimizes hardware redesign for different communication speeds, promoting efficient use of hardware resources and reducing development time and electronic waste.

9 Conclusion

The Verilog-Based UART Transmitter with Configurable Baud Rate Generator was successfully designed, implemented, and verified using EDA Playground. The proposed system accurately converts 8-bit parallel data into serial data by following the standard 8N1 UART communication protocol. The configurable baud rate generator provides flexibility to operate at different communication speeds without modifying the overall architecture.

Simulation results confirmed the correct transmission of the start bit, data bits, and stop bit with accurate timing. The modular design improved the readability, scalability, and ease of verification of the system. Overall, the project provides a reliable and efficient UART transmitter that can be used in FPGA-based and embedded communication applications while serving as a strong foundation for future enhancements, such as the addition of a UART receiver, parity checking, or FIFO buffering.

Bibliography

[1] Samir Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd Edition, Prentice Hall, 2003.

[2] M. Morris Mano and Michael D. Ciletti, *Digital Design*, 6th Edition, Pearson, 2018.

[3] Stephen Brown and Zvonko Vranesic, *Fundamentals of Digital Logic with Verilog Design*, 3rd Edition, McGraw-Hill Education, 2014.

[4] Douglas L. Perry, *VHDL and Verilog Compared and Contrasted Plus Modeled Example Written in VHDL, Verilog and C*, Springer, 2002.

[5] Jan M. Rabaey, Anantha Chandrakasan, and Borivoje Nikolic, *Digital Integrated Circuits: A Design Perspective*, 2nd Edition, Prentice Hall, 2003.

[6] Wayne Wolf, *FPGA-Based System Design*, Prentice Hall, 2004.

[7] IEEE Standards Association, *IEEE Standard for Verilog Hardware Description Language (IEEE Std 1364-2005)*, IEEE, 2006.

[8] EDA Playground, *Online Verilog/VHDL Simulator*, Available at: `https://www.edaplayground.com`

[9] Texas Instruments, *UART Communication Basics*, Available at: `https://www.ti.com`

[10] AMD Xilinx, *Vivado Design Suite User Guide*, Available at: `https://www.xilinx.com`

[11] Intel FPGA, *Quartus Prime Software User Guide*, Available at: `https://www.intel.com/fpga`

[12] Nandland, *UART Serial Port Module Tutorial*, Available at: `https://nandland.com`